Automated Sage 50 Invoice Posting & ERP Integration

Desktop automation utility for daily batch invoice posting into Sage 50 Accounts

The client was entering invoices into Sage 50 Accounts by hand, every day — repetitive, error-prone, and it had to happen whether or not the usual person was in. This project replaced that with a scheduled desktop utility that posts the previous day's batch automatically, and that the accounts team can reconfigure themselves without a developer.

90%+	0	Daily	90
MANUAL EFFORT REMOVED	CONFIG FILES TO EDIT	UNATTENDED RUN	DAYS POST-LAUNCH SUPPORT

MY ROLE

Full-Stack Developer — desktop application, posting engine, deployment

THE PROBLEM

- **Repetitive manual entry.** A person posting the same batch of invoices into Sage 50 every working day.
- **Authentication obstacles.** Sage portal access involved two-factor prompts and CAPTCHA challenges that an unattended process has to survive — or fail loudly rather than silently.
- **Non-technical owners.** The people who needed to change the schedule, the date range or the login were accountants, not engineers. Editing a JSON file was not an acceptable answer.

WHAT I BUILT

Configuration interface. A Windows desktop panel where staff set the run time, the date range to post, and the Sage credentials — with a "test login" action so a wrong password is caught at the desk rather than at 2am. No raw config files, no code.

Posting engine. A background process that connects to Sage 50 through its COM API, reads the invoices for the configured period, batches them, and posts them to the ledger.

Logging and failure handling. Every run writes a readable log: what it connected to, how many invoices it found, what posted, what didn't and why. Authentication lockouts surface as a clear prompt instead of a silent stall — the failure mode that matters most in an unattended job.

Scheduled, unattended execution. Wired into Windows Task Scheduler for a daily trigger, deployed directly into the client's live environment with executable whitelisting configured.

The decision that mattered most: putting the parameters behind a UI rather than in a config file. It is the difference between a tool the client owns and a tool that needs me every time a date range or a password changes. Automation that only the developer can reconfigure is not finished automation.

TECHNICAL STACK

AREA	TECHNOLOGY
Application and engine	C# / .NET
User interface	WPF Windows desktop GUI
ERP integration	Sage 50 Accounts COM API / SDK
Scheduling	Windows Task Scheduler, daily unattended trigger
Deployment	Client live environment, executable whitelisting, remote desktop access

OUTCOME

- **The daily batch runs itself.** Invoices post on schedule with no one present.
- **The client owns the controls.** Schedule, dates and credentials are changed by accounts staff, in a form, without a developer in the loop.
- **Failures are visible.** Logs and prompts surface a problem the same morning, rather than at month-end reconciliation.
- **Delivered into production** with whitelisting, handover and 90 days of post-launch support.